

Network Services for the Support of Very-Low-Resource Devices

Thomas Fuhrmann, Till Harbaum, and Martina Zitterbart
Institut für Telematik, Universität Karlsruhe (TH), Germany

Abstract—Visions of future computing scenarios envisage a multitude of very-low-resource devices linked by power-efficient wireless communication means. This paper presents our vision of such a scenario. From this vision requirements are derived for an infrastructure that is able to satisfy the largely differing needs of these devices. The paper also shows how innovative, collaborating applications between distributed sensors and actuators can arise from such an infrastructure. The realization of such innovative applications is illustrated with two examples of straightforward services that have been implemented with the AMnet infrastructure that is currently being developed in the FlexiNet project. Additionally, first performance measurements for one of these services are given.

Index terms — Bluetooth, Programmable networks, Sensor-actuator networks

I. INTRODUCTION

Bluetooth has the potential to become a widely used wireless access technology for very-low-resource devices, like wireless head-sets, wearable computing gadgets, and various sensor-actuator-components in the field of ambient computing. Among the key advantages of such devices are the low power consumption and their small physical size. Important drawbacks are limited programmability, CPU performance and memory resources: Either the devices do not offer programmability at all (e.g. wireless head-sets) or they offer considerably lower computing power than similar 'large-footprint' devices (e.g. a Bluetooth Palm-PDA as compared to a WaveLAN IPAQ).

On the other hand, adaptability and programmability would greatly enhance the use of such devices. E.g., a head-set could be updated to work with new data-formats and communication protocols. A Palm PDA could access multimedia Internet content, originally created for much more powerful machines. Off-the-shelf sensors and actuators could be integrated into, e.g., a specific building automation system without having to be tailored for that use. In other words, there is a need also for very-low-resource devices to become programmable.

In the original scenario where Bluetooth was designed to replace the cabling between one specific main component (desktop-PC, mobile phone) and an attached peripheral, this limitation did not matter much. All the required adaption could be performed by the desktop-PC, the mobile phone, etc. In a ubiquitous, pervasive, or ambient computing scenario, where devices move around freely, this limitation matters greatly, since *there is no* single component that is in charge of preparing the data for *its* Bluetooth peripheral.

In this paper, we discuss one possible solution to this challenge, the use of programmable network methods to provide flexible data adaptation for very-low-resource mobile devices. This idea is not new as such. It has already been discussed as an approach to improve the performance of various mobile devices, e.g., proxies for the Internet access of mobile phones, etc. But to the best of our knowledge it has not been studied in the context of very-low-resource devices (sensor-actuator networks, etc.), a context that has considerably different requirements and is certainly worth being studied. Additionally, so far we are not aware of practical experiments that would allow a realistic judgment of the feasibility of the programmable network approach for this context.

This paper addresses both issues: Firstly, it gives an overview of our on-going research activities aiming at the combination of network services [6] with very-low-resource Bluetooth devices. Secondly, alongside this conceptual work, the paper also presents first results that indicate that this approach is indeed promising with respect to flexibility, manageability, and achievable throughput. This paper is structured as follows: Section II describes two guiding scenarios that serve as foundation for deriving requirements for an infrastructure supporting very-low-resource mobile devices in an ambient or ubiquitous computing setting. Section III explains the architectural structure of our solution. It describes the programmable network system architecture that forms the basis for our on-going research work. Sections IV and V present two examples for services in such smart environments. Section VI gives a brief overview of related work. Finally, section VII concludes with an outlook to future work.

II. TWO SCENARIOS

Our studies consider the possibility to create a smart ambient computing environment that links a large number of very-low-resource devices. The term *very-low-resource device* shall be understood as a computing device that operates below a level of a few million instructions per second, that has a few hundred bytes of random access memory only, and that consumes at most one to two hundred milliwatts (including sensors and radio) during peaks of operation, i.e., we consider such devices to typically contain an 8 bit microcontroller.

Two important guiding scenarios for the use of such devices are the *disintegrated user interface* and *collaborating embedded devices*.

A. The disintegrated user interface

At the heart of our studies lies the vision of a user interface that is separated into the different modal aspects of human-computer interaction: an ear-phone, a clip-on microphone, eventually a head-mounted display, and the BlueWand, a small, pen-like device that can detect its position and movement in space [8]. These devices communicate (e.g., via Bluetooth) with other devices either carried around by the respective person, or with the various embedded devices in their vicinity: consumer electronics, building automation, etc.

Such a setting would allow many amenities: The audio of the TV-set could follow us when we leave the living-room to fetch another beer. If we stayed away longer, we could take a lightweight PDA out of our pocket and watch an interesting scene there. Spoken commands can be used to switch the channel of the TV and to adjust the air-condition. Gestures (with a BlueWand) switch on the lights while biometric identification extracted from our gestures and our voice proves us to be the legitimate person to do so (and not a burglar).

This vision is ambitious, not so much because the devices themselves are, but because this vision requires very different tiny devices to interact with various, greatly differing potential peers. With this vision, it is *not* sufficient to link each of the devices to one rather fixed master device as is the case in the original Bluetooth scenario that aimed at the *head-set to mobile phone* example. It is also not sufficient to provide mere network connectivity, as is the case in the typical wireless ad-hoc scenario where rather heavy-weight general purpose machines run (almost) full-sized network applications on top of standard protocol stacks. With our vision it is not possible to use the same protocols and data-formats for all applications: A Bluetooth headset has other requirements than a PDA whose requirements again differ from those of a TV-set. Hence, the devices need more than mere connectivity: *They need network services that provide them with content in their desired individual format.*

B. Collaborating embedded devices

Bluetooth has the potential to provide inexpensive wireless communication for everyday objects: reconfigurable door-plates, context aware coffee-cups, smart flowerpots, etc. Combined with building automation (lights, air-conditioning, blinds, access control, etc.) and white goods (washing machine, fridge, etc.) this yields the potential to create a smart office or house.

In this scenario, too, the increased benefit is possible when arbitrary devices can profit from one another: Imagine, e.g., a flowerpot that can close the blinds to avoid sun-light burning the plant. Imagine a door-plate that indicates where the respective person can be found (“Tom is chatting in the meeting room”) or that you can dictate messages to (“Tom, please call me, it’s urgent. - Tim”). Even though using state-of-the-art Bluetooth equipment for medical purposes is not advocated, the same principle can easily be extended towards such uses: E.g., when an electrocardiographic sensor detects an indication for a heart attack, it triggers an emergency call *and* unlocks the door to allow access for the paramedics.

Here, again, direct communication alone does not suffice. Neither the clip-on microphone nor the door-plate will — in

foreseeable future — be capable of speech recognition that could transcribe arbitrary sentences. Moreover, considering the standardization battles in industry and the role-out speed of fancy gadgets, it is very unlikely that all the sensors and actuators participating in one of the described operations will be equipped with the same protocol. But people will doubtlessly expect everyday objects to work straight away without (much) configuration. They expect to be able to just walk into the next do-it-yourself store and buy a flowerpot that knows about their plant. They will not want to put an accompanying CD-ROM into their PC, install FlowerPlant 2.0 and see it crash their word-processor application, or, even worse, have their spread-sheet calculation kill their pot-plant.

With other words, all the described aspects require a third party to mediate between these devices (flowerpot and blinds, doorplate and microphone).

III. A NETWORKING PLATFORM

As has been explained above, we believe that neither of the devices participating in one of the described smart settings can perform computationally expensive tasks. Also, we do not expect that these devices can be adapted to support all the data-formats and protocols that might be required for a specific application. Furthermore, we do not believe that the typical desktop PC (understood as user-administrated central communication hub) is an adequate place for such control and mediation tasks.

Given that, the next obvious place to provide such functionality is the network. By this, we mean that the network’s mere connectivity aspect is supplemented by the network’s ability to perform computational tasks required by the very-low-resource devices accessing the network.

Programmable networking architectures pursue exactly this goal, i.e. to facilitate the rapid and flexible introduction of new services into a communication network. Other mechanisms like proxies with automated software update could achieve similar results. This paper does hence only argue that programmable networks are *a* solution to the problem, not *the* solution.

The rest of this section gives a brief introduction into those aspects of our platform that are important to the topic discussed here. Readers familiar with AMnet might want to skip to the next section that describes how AMnet can be used for the scenarios considered here.

The AMnet programmable networking architecture has been developed over the last couple of years [14], [6], [7]. Currently, it is further enhanced in the *FlexiNet* project¹. This project investigates the applicability of programmable networks. It does so particularly by the prototypical development of real-world service examples that go beyond the toy-examples often used in active and programmable networking.

The AMnet approach is centered around the concept of *service modules*, small pieces of code that can be drawn from administratively managed *service module repositories* (see figure 2). These modules can tap into the packet stream flowing

¹Funded by the German Ministry of Education and Research, BMBF, under grant number 01AK019A

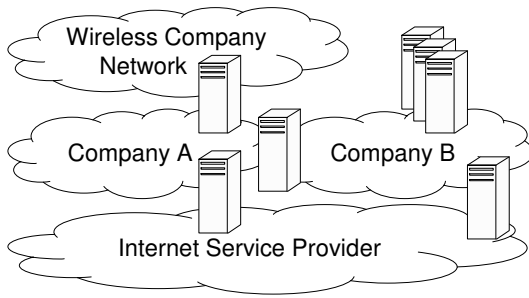


Fig. 1. AMnet deployment scenario: Active nodes serve as gateways between small sub-networks or provide services as an off-path cluster. — The active nodes are *not* assumed to be deployed near the core network!

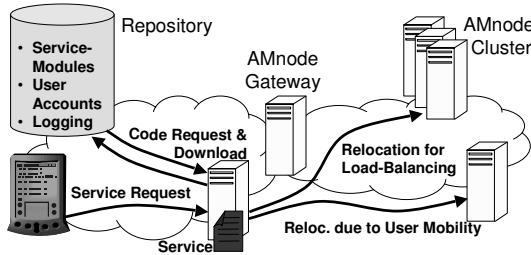


Fig. 2. Usage example: A PDA signals its service request into the network. An AMnode catches the request and retrieves the corresponding code. If the PDA moves to another sub-network, the service module will be relocated accordingly.

through the nodes of the programmable network, process and modify the packets, and either re-inject the modified packets into the network layer or hand them over to other modules running on that node.

Several components of the AMnet node software provide the functionality for such an active node, a so-called AMnode (cf. fig. 3): The *execution environment* hosts the service modules and controls their interaction with the node's network layer, the chaining of the modules, and their interaction with various other helper functions that wrap, e.g., typical API functions of the underlying operating system. The *evaluation* and *relocation* mechanisms select the node(s) where a service is to be executed. If conditions change either in the programmable network or by, e.g., end-device mobility, the relocation mechanism can move a service to another node, if required also while the service remains active. The *resource monitor* keeps the node under surveillance such that services cannot use resources beyond their assigned quotas.

AMnet conceives two major types of services: application services and infrastructure services.

1. *Application services* are initiated and controlled by (applications running on) the end-devices. To this end, these devices need to interact with the AMnet signaling. Depending on the kind of service, the signaling device need not be affected by the service directly, but benefit only indirectly.

Example: The flowerpot invokes a service that taps into the data stream generated by the light and temperature sensors. As a result this service controls the air-conditioning and blinds according to the plant's needs.

2. *Infrastructure services* are launched by the network administrator. Such a service can run quietly and become active only upon certain conditions in the network.

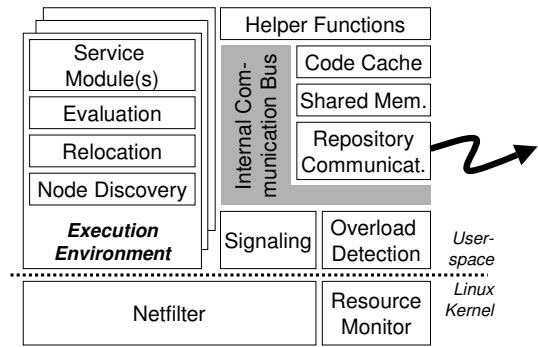


Fig. 3. AMnet active node architecture

Example: An intrusion detection service surveys the traffic between the various devices. Thereby, it can detect attacks on the data infrastructure and launch appropriate blocking services. Unlike most networking scenarios, here, intrusion detection can also be applied to physical intrusion of, e.g., burglars, by detecting suspicious traces in the sensors' data-streams. The very same principle can be used to provide sanity checks in a complex sensor-actuator network.

Depending on the requirements of a particular service, it needs to be run on one or multiple nodes within the network. For some services, it is sufficient to find only one AMnode somewhere in the network. Other services might require a few more nodes. By means of redirection, a resource-consuming service can be placed on a more powerful node to which the required data-stream is then redirected (see figure 2).

The evaluation and relocation mechanisms are essential for this placement and movement of services. Each service can determine the resources it needs for operation. Resources are, e.g., computing or memory resources, but also the data-streams that a service needs to operate on. When a service is invoked, a small, service-specific piece of code checks which nodes provide all the required resources. Depending on the requirements several evaluation strategies can be pursued (see [9] for details).

Relocation, on the other hand, provides additional means to move a running service to another node [10]. This might be for optimization or load-balancing. Most typical in the scenarios considered here, relocation will be used to take code along with a mobile end-device.

AMnet is supplemented by a management system that stands besides the active nodes. With the help of this management system, AMnet can provide authentication, authorization, and accounting functionality: Services can be restricted to certain users, applications, or devices. The use of a service is accounted, e.g., for billing purposes.

The management system is organized into domains that can be stacked hierarchically. So, e.g., a facility manager can control the aspects she wants to control herself and delegate all other aspects to a provider of her choice. The same organization also ensures that a service vendor that chose to provide its service on a per-use basis will receive the corresponding accounting data. Finally, the management system hierarchy han-

dles code updates for the service modules. This ensures that updates and patches are deployed quickly.

For a more detailed description of the AMnet architecture please refer to [6].

IV. VIDEO STREAMING TO VERY-LOW-RESOURCE DEVICES

Video streaming is an often studied example for the tight interplay of communication and multimedia processing. Typically, there is a trade-off between compression and required processing resources, i.e., the better the compression ratio the higher the complexity of the decoding process. The choice where to operate in this trade-off depends on the capabilities of the respective device: A modern desktop PC can decode MPEG video in software, but a Palm PDA can not. Accordingly, video messages on such PDAs have to use other compression schemes.

There is a second trade-off, namely between data-rate and quality. This is the well-known trade-off typically addressed by transcoders and layered encoding schemes. (See, e.g., [18] for a discussion of layered encoding schemes versus transcoding services.) We do not consider this trade-off here, but rather discuss how a very-low-resource device can access data-streams not specifically authored for such a device. This is different from the usual data-rate and quality trade-off.

Consider, e.g., digital video broadcast (DVB) which has become so popular that respective kernel support will be integrated into regular Linux 2.6. Together with a Video LAN client it is now possible to view live video streams and recorded video on a desktop PC or even an IPAQ PDA. But no means are provided to easily access this data stream from a very-low-resource device like a Palm PDA that is far from displaying an MPEG video stream natively.

One straightforward solution might be to provide the content in a suitable format. But this is not always feasible in practice. Mass-market formats like DVB and DVD use MPEG2. Alternative formats are not easily established. Moreover, different end-devices often have largely differing capabilities because different vendors often decide to exploit different regions of the design space. Hence, a coding scheme that would suit all devices (not only those from one vendor or those based on one technology) is very unlikely to be found. As a consequence, this scenario requires the use of a transcoder.

To model this scenario, a Palm m515 PDA equipped with a Toshiba Bluetooth card and an MPEG2 video streaming source (DVB TV-channel or DVD) were used. A schematics of the demonstration is shown in figure 4. A photograph of the running application is presented in figure 5. The same MPEG2 video stream is simultaneously displayed by two devices, a desktop PC and the Palm. While the desktop PC can display the original stream in full quality, the Palm needs the help of an external transcoder.

Please note, that this is an extreme demonstration set-up, since the Palm Bluetooth implementation that we use is only capable of transferring 100 kBit/s. Furthermore, the Palm's CPU (33 Mhz 68VZ328) is incapable of performing complex decoding operations — as would be required for MPEG processing — at a sufficient rate. Therefore, a much simpler video format



Fig. 5. Photograph of a Palm PDA using the AMnet video transcoding service

has be used on the PDA. It makes direct use of the Palm's video settings, e.g., color space, etc.

As long as the Palm is switched off, the MPEG video data flows through the AMnode untouched, i.e., while being idle, an AMnode operates completely transparently and does not modify the data it is forwarding at all. It is passively waiting for service requests. When the Palm is switched on it connects to a Bluetooth access point (which is in our current setup just the next desktop PC). This requires a Bluetooth discovery to find all Bluetooth devices in the vicinity (see figure 6). This might include other Bluetooth equipped PDAs, cell-phones and PC peripherals. In a second step, the Palm then uses the Bluetooth service discovery protocol (SDP) to check which of the devices actually offer AMnet network connectivity. This step excludes devices like cell-phones and PC peripherals. In a final step, the Palm establishes a connection and requests the video stream.

This request triggers the evaluation procedure. It finds an AMnode which is on the data-path and can transcode the video to the required Palm format. The AMnode retrieves the code for the service from the repository and starts an execution environment for this service. In order to provide the PDA with the desired video, the service module has to perform several

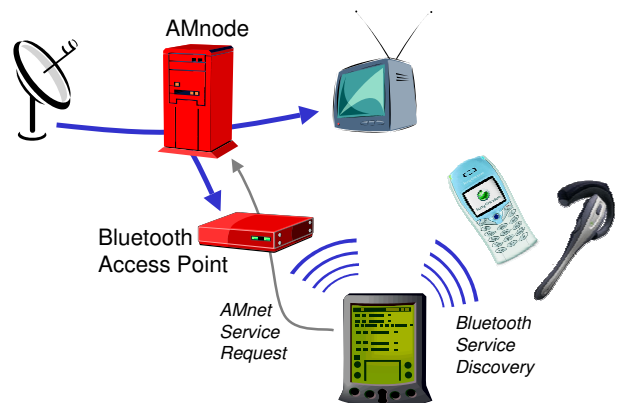


Fig. 6. Discovery of Bluetooth devices and AMnet services

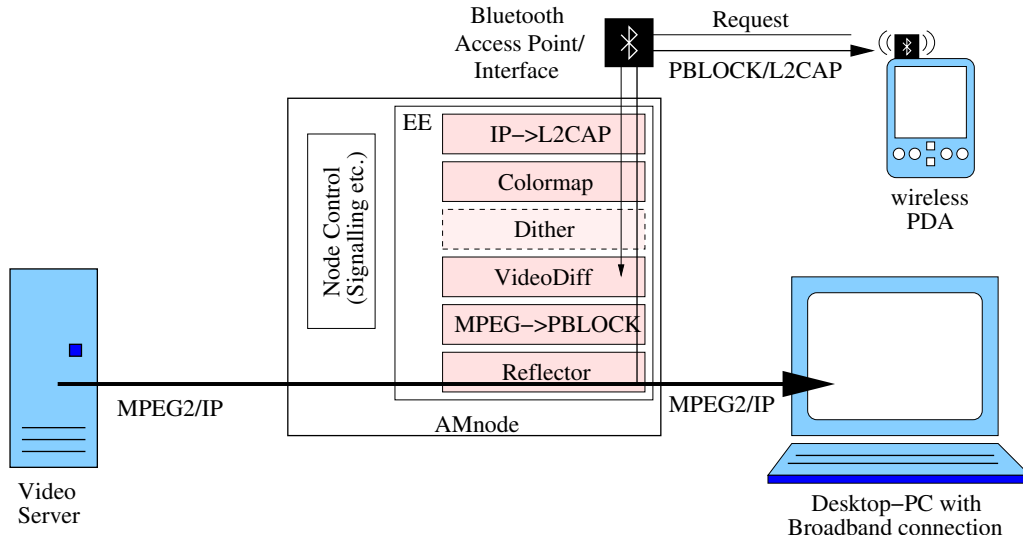


Fig. 4. Schematics of the AMnet video transcoding service for Bluetooth PDAs

operations inside the AMnode:

- The existing video stream has to be split into one stream being forwarded to the existing clients and one stream being redirected to the Palm. (This part of the service has to be executed by an on-path AMnode.)
- The video size (and hence its resolution) has to be reduced in order to meet the needs of the low resolution PDA screen and not to waste wireless bandwidth.
- The video transmission rate has to be dynamically adapted to the current bandwidth of the wireless link.
- The video has to be adopted to the eight bit colormap used by the Palm OS operating system.
- The video has to be transcoded from IP based transmission to the L2CAP protocol of the Bluetooth stack. (This part of the service has to be executed by the access point. Since we use an AMnode as access point, we were able to implement this part as an AMnet service as well.)

Even though it would have been possible to implement all these operations within one AMnet service module, the flexible concept of chaining service modules additionally allows a modular implementation. Thereby, the system is opened for future expansions and re-use of individual functionalities is made possible. In the example, one module could be re-used from a prior setup, four modules had to be written additionally:

- The *reflector module* splits an existing data stream and allows a multicasting like transmission of one source stream to several receivers. This module had already been implemented for the MP3 multicast reflector, described in [5], i.e. it could be re-used.
- The *MPEG to PBLOCK transcoder module* translates the MPEG2 data stream into the proprietary PBLOCK format used on the Palm. This transcoding generates comparatively much load on the node.
- The *videDiff module* determines differences between video blocks and, on request by the receiver, initiates the transmission of the video block with the biggest difference in a certain time span. By operating on request of the re-

ceiver this module realizes flow control for the PDA.

- The *colormap module* then translates the video blocks from the RGB color space into the PDA's indexed color space.
- The *IP to L2CAP module* finally terminates the IP processing and forwards all data to the Bluetooth link.

When this set-up was first tested, it became obvious that an additional step could greatly increase the achievable quality, namely a step that avoids the reduction of the video to only 256 colors by *dithering*. Fortunately, the flexible concept of AMnet service modules allowed to dynamically integrate an additional module into the module chain. Now, the additional *dither module* applies a simple dithering algorithm to each processed video block, causing the processing of color-mapping to work significantly better.

This scenario has shown several interesting advantages of the AMnet concept: The transparent integration of AMnet technology into the existing forwarding path allows the transparent deployment of new services without effecting the existing end-systems. Packet video streams can be specifically adapted to the requirements of a specific end-device. Here, the Bluetooth access point, an AMnode, adapts the video to the Palm's color palette. Furthermore, the module concept of AMnet allows the re-use of existing technology as well as the flexible introduction of new functionality whenever necessary. A measurement of the service's performance is shown in figure 7. It demonstrates that such a service is in fact feasible on, e.g., access points that are active nodes.

V. THE FLOWERPOT EXAMPLE

The previous example has demonstrated how an active node can adapt data-formats to suit the needs of a specific end-device. This section illustrates how programmable network services can create collaboration among very-low-resource devices. To this end, we consider the following setting of wirelessly connected sensors and actuators (see also figure 8):

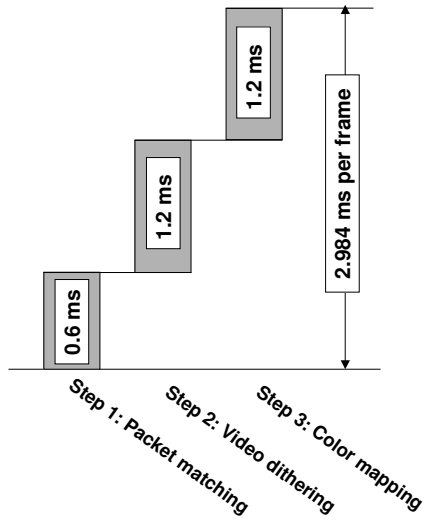


Fig. 7. Performance measurement of the Palm video example: The Bluetooth access point (an 800 MHz Pentium AMnode) transcodes a packet video stream to specifically suit the Palm video color palette.

- A flowerpot, equipped with sensors to measure the humidity and temperature of the soil in the pot. (Currently, we employ a capacitive measurement method that has an accuracy of ≈ 5 ml when filled with water.)
- Clip-on photometric sensors that measure direct and indirect light intensity on the spot where they are attached to. (An additional accelerometer detects movements of the clip.)
- A watering can that measures the amount of water that is applied to a flower-pot.
- Blinds, lights, and air-conditioning that can control the conditions in the room.

We assume that the devices use the same layer-two protocol, e.g., Bluetooth. But we *do not assume* that they are otherwise specifically configured to collaborate. I.e., we imagine people to just stop by a do-it-yourself shop to buy a device that then automatically integrates into the smart environment without further configuration. To achieve this, the devices need an infrastructure that mediates the desired collaboration between the devices.

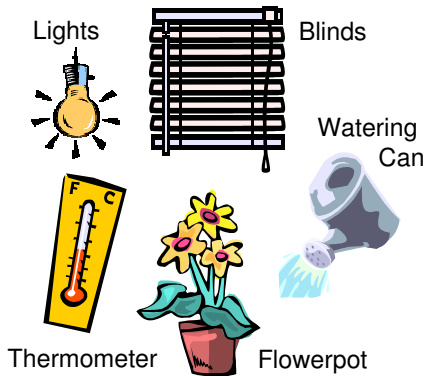


Fig. 8. The flowerpot example: everyday objects that collaborate without configuration

To illustrate this, consider the user buying a new photometric clip-on sensor. She places it at the rod holding her plant. As a first step, this sensor automatically integrates into its smart environment: It issues photometric measurements. These measurements are caught by a service that controls the pot-plants of the room. It does so for it is interested in knowing the light intensity in the room. Other services that are also interested in such measurements can do the same. These services can decode the data-format of the clip since they can draw the appropriate module from the repository system.

After some time, enough correlation data has been acquired from the photometric sensor, the blinds and the watering can that the service can identify the sensor with a specific pot-plant. This correlation occurs because the light intensity decreases whenever the corresponding blinds close, it increases when these blinds open. Watering the plant increases the humidity of the soil in the flowerpot, and at the same time the accelerometer of the photometric clip-on detects activity. All these correlation patterns yield evidence on the physical layout of the sensors and actuators in the room. This data can automatically be collected and processed by the services in the programmable network.

When the relations between the different devices have been discovered, the services can create new collaborations: When the user is absent and the flowerpot service detects too much sun-light entering the room it can close the blinds. It can send a message to the user to remind her of watering the plant, and it can keep her from watering the plant too much, etc.

The charme of this scenario is, that at any point in time both, new services and new sensors or actuators, can be introduced in order to improve the existing system. The user can buy a new sensor, e.g., the photometric clip to keep the plant from being burnt. And the user can subscribe to a new service that uses the existing sensors, e.g., a burglar warning that combines the accelerometer data from various sensors to detect unauthorized people climbing through the window and involuntarily moving the plant.

There are plenty of applications for such network services that make use of the sensors and actuators of the very-low-resource devices that are about to populate homes and offices soon.

VI. RELATED WORK

Open and programmable networking has been a field of intensive research for quite a while. Therefore, it is impossible to give a brief overview without being forced to arbitrarily choose among the many approaches in this field. We will hence only exemplarily present a few projects to place AMnet in context. Please see, e.g., [23], [3], [4] for a general overview of fundamental concepts of active and programmable networks and [14], [6] for an overview of the AMnet architecture and its improvements that were developed recently.

AMnet's active nodes are Linux based software routers. A similar approach has been pursued by the Click Modular Router project [17]. There, low-level extensions to the regular network protocol stack provide a router environment in which so-called *elements* perform the basic processing steps like packet classification and mangling. Compared to AMnet, which (mostly) runs

in the user-space of an unmodified Linux installation, Click's direct hardware access trades security and programming ease against performance. Both projects' common objective, namely to benefit from existing operating system functionality, is also shared by SILK [2], in which a port of Scout [20] replaces the standard Linux protocol stack.

Similar ideas have also been developed outside the active and programmable networking community, see, e.g., [12] and references therein.

During the recent years, sensor networks have become an area of intensive research. [24] proposes a taxonomy for these networks. Especially questions in the lower layers are thoroughly studied: energy saving, medium access, routing protocols, etc. For a recent survey see [1]. The same holds for questions in mobile ad-hoc networks, see, e.g., [16], and for smart devices [22], [13], [11], [15]. But comparatively few authors consider the network service aspects addressed here. [21] gives a brief overview of middleware aspects in sensor networks. [25] studies declarative queries in networks of very-low-resource devices. [19] describes a system for habitat monitoring.

VII. CONCLUSION AND OUTLOOK

This paper has described two scenarios where network services can support very-low-resource devices, namely the *dis-integrated user interface* and *collaborating embedded devices*. Both scenarios are characterized by the fact that the participating devices do not have sufficient resources to perform data-format or protocol transformations, or to execute applications that could make integrative use of these devices. In order to circumvent this limitation we have proposed the use of network services executed on the active nodes of a programmable network. A detailed example of such a service was given, namely the transparent adaptation of a video stream to a Palm PDA that has been implemented with the AMnet system. Performance measurements show that such a service is practically feasible. To illustrate the future potential of network services for very-low-resource devices we then described the smart flowerpot and its collaboration with other smart devices. We believe that such scenarios, too, can greatly benefit from services that run on active nodes, e.g., deployed in form of access points for Bluetooth devices, etc.

Currently, we are building various prototypes of such smart devices, including the smart flowerpot. We are confident to soon be able to present further performance studies and experiences with these settings.

REFERENCES

- [1] Ian F. Akyildiz, Weilian Su, Yogesh Sankarabramanian, and Erdal Cayirci. A survey on sensor networks. *IEEE Communications Magazine*, 40(8):102–114, August 2002.
- [2] Andy Bavier, Thimo Voigt, Mike Wawrzoniak, Larry Peterson, and Per Gunningberg. SILK: Scout paths in the Linux kernel. Technical Report 2002-009, Uppsala Universitet, February 2002.
- [3] Kenneth L. Calvert, Samrat Bhattacharjee, Ellen Zegura, and James Sterbenz. Directions in active networks. *IEEE Communications Magazine*, 36(10):72–78, October 1998.
- [4] Andrew T. Campbell, Herman G. De Meer, Michael E. Kounavis, Kazuho Miki, John B. Vicente, and Daniel Villela. A survey of programmable networks. *ACM SIGCOMM Computer Communication Review*, 29(2), April 1999.
- [5] Thomas Fuhrmann, Till Harbaum, Panos Katsianidis, Marcus Schöller, and Martina Zitterbart. Network services with FlexiNet. Technical report, Institut für Telematik, Universität Karlsruhe, 2002.
- [6] Thomas Fuhrmann, Till Harbaum, Marcus Schöller, and Martina Zitterbart. AMnet 2.0: An improved architecture for programmable networks. In *Proceedings of IWAN'02*, Zürich, Switzerland, December 2002.
- [7] Thomas Fuhrmann, Panos Katsianidis, Marcus Schöller, and Martina Zitterbart. Results on the practical feasibility of programmable network services, 2003. Submitted.
- [8] Thomas Fuhrmann, Markus Klein, and Manuel Odendahl. The bluewand as interface for ubiquitous and wearable computing environments. In *Proceedings of 5th European Personal Mobile Communications Conference (EPMCC)*, Glasgow, Scotland, April 22-25 2003.
- [9] Thomas Fuhrmann, Marcus Schöller, Christina Schmidt, and Martina Zitterbart. A node evaluation mechanism for service setup in AMnet. In *Proceedings of Kommunikation in Verteilten Systemen (KiVS)*, pages 29–37, Leipzig, Germany, February 26-28 2003.
- [10] Thomas Fuhrmann, Marcus Schöller, and Martina Zitterbart. Service relocation in programmable networks, 2003. Submitted.
- [11] Hans-W. Gellersen, Albrecht Schmidt, and Michael Beigl. Adding some smartness to devices and everyday things. In *Proceedings of IEEE Workshop on Mobile Computing Systems and Applications (WMCSA'00)*, Monterey, USA, December 2000. IEEE Press.
- [12] Manuel Günter and Torsten Braun. Service delivery control with mobile code. In *Proceedings of the IFIP Conference on Intelligence in Networks (SmartNet 2000)*, September 2000.
- [13] Albrecht Schmidt H.-W. Gellersen and Michael Beigl. Multi-sensor context-awareness in mobile devices and smart artifacts. *Journal on Mobile Networks and Applications, Special Issue on Mobility of Systems, Users, Data and Computing in Mobile Networks and Applications (MONET)*, 7(5), October 2002.
- [14] T. Harbaum, A. Speer, R. Wittmann, and M. Zitterbart. Providing Heterogeneous Multicast Services with AMnet. *Journal of Communications and Networks*, 3(1):46 – 55, March 2001.
- [15] Lars Erik Holmquist, Friedemann Mattern, Bernt Schiele, Petteri Alahuhta, Michael Beigl, and Hans-W. Gellersen. Smart-its friends: A technique for users to easily establish connections between smart artefacts. In *Proceedings of UbiComp 2001*, Atlanta, Georgia, September 2001.
- [16] Xiaoyan Hong, Kaixin Xu, and Mario Gerla. Scalable routing protocols for mobile ad hoc networks. *IEEE Network*, 16(4):11–21, Jul/Aug 2002.
- [17] Eddie Kohler, Robert Morris, Benjie Chen, John Jannotti, and M. Frans Kaashoek. The click modular router. *ACM Transactions on Computer Systems*, 18(3):263–297, August 2000.
- [18] Jiangchuan Liu, Bo Li, and Ya-Qin Zhang. Adaptive video multicast over the internet. *IEEE Multimedia*, 10(1):22–33, January-March 2003.
- [19] Alan Mainwaring, David Culler, Joseph Polastre, Robert Szewczyk, and John Anderson. Applications and os: Wireless sensor networks for habitat monitoring. In *Proceedings of the first ACM international workshop on Wireless sensor networks and applications*, September 2002.
- [20] David Mosberger. *Scout: A Path-based Operating System*. PhD thesis, Department of Computer Science, University of Arizona, July 1997.
- [21] Kay Römer, Oliver Kasten, and Friedemann Mattern. Middleware challenges for wireless sensor networks. *ACM SIGMOBILE Mobile Computing and Communication Review*, 6(2), 2002.
- [22] Albrecht Schmidt and Kristof Van Laerhoven. How to build smart appliances. *IEEE Personal Communications*, August 2001.
- [23] David L. Tennenhouse, Jonathan M. Smith, W. David Sincoskie, David J. Wetherall, and Gary J. Minden. A survey of active network research. *IEEE Communications Magazine*, 25(1):80–86, January 1997.
- [24] Sameer Tilak, Nael B. Abu-Ghazaleh, and Wendi Heinzelman. A taxonomy of wireless micro-sensor network models. *ACM SIGMOBILE Mobile Computing and Communications Review*, 6(2):28–36, April 2002.
- [25] Yong Yao and Johannes Gehrke. The cougar approach to in-network query processing in sensor networks. *ACM SIGMOD Record*, 31(3), September 2002.